Today
Welcome to your next lesson: Dict of lists and grouping by key.

I'm preparing your personalized lesson using what you've learned so far, what comes next, and the best ways to help you practice.

Step by step, you're moving forward.

Imagine we run a small shelter for magical pets 🐉🦄

We have data like this, one pet per record:

Python

```
pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Spark", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
]
```

This is a list of dicts: each pet is its own little dictionary in a big list.

Now suppose we often need: "Give me all dragons" or "all unicorns" fast.

Another way to store data is a dict of lists:

Python

```
by_species = {
    "dragon":  ["Flame", "Spark"],
    "unicorn": ["Misty"],
}
```

Here each key is a species, and the value is a list of pet names in that species — like buckets.

Quick gut check:

If you had to answer "list every dragon's name" many times per minute, which structure feels more natural for that job?

A) The list of dicts (pets = [...])
B) The dict of lists (by_species = {...})
B

What do you think? Pick the option that best matches your reason 👇

You picked B) dict of lists for answering "list every dragon's name" many times. Why is that usually a better fit?

A.

Because each species is a key, so we can jump straight to that list of names.

B.

Because lists are always faster than dictionaries in Python.

C.

Because we can only store one pet in each species bucket.

D.

Because dictionaries automatically sort the pets by name.

Exactly right 🎯

Because each species is a key, a dict of lists lets us jump straight to "dragon" and grab that whole list in one step, instead of scanning every pet and checking its species over and over. It’s like having labeled bins: go straight to the dragon bin and pull out all the names.

Next step: I want to show you how to build that by_species dict incrementally from the flat pets list, one pet at a time.

Are you up for a short, step-by-step example of that?

Yes, show me!

Nice 👀 Let’s go very step by step.

We’ll start with this flat list again:

Python

```
pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Spark", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
]
```

Goal: build this gradually:

Python

```
by_species = {
    "dragon":  ["Flame", "Spark"],
    "unicorn": ["Misty"],
}
```

First idea: we’ll walk through pets one pet at a time and keep a dict called by_species that starts empty:

Python

```
by_species = {}
```

Then for each pet, we’ll:

Look at its "species"
Add its "name" into the correct list for that species

I’ll show you the exact code next, but first: does the plan make sense (empty dict → process pets one by one → fill species buckets)?

Yes, that makes sense

Great 👍 Let’s turn that plan into concrete code, one line at a time.

Here’s the full pattern first:

Python

```
by_species = {}

for pet in pets:
    species = pet["species"]
    name = pet["name"]

    if species not in by_species:
        by_species[species] = []

    by_species[species].append(name)
```
Focus on just this tiny piece for now:

Python

```
if species not in by_species:
    by_species[species] = []
```
This means:

If we don’t have a bucket for this species yet
Create an empty list [] for that key in the dict

So the very first time we see "dragon", we make by_species["dragon"] = [].

Does that small step — “create an empty list the first time we see a species” — feel clear so far?

its clear

Perfect ☑

Now let’s see the whole flow with a tiny dry run, so you can see the buckets filling.

Starting state:

Python

```
pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Spark", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
]

by_species = {}
```
We loop:

Python

```
for pet in pets:
    species = pet["species"]
    name = pet["name"]

    if species not in by_species:
        by_species[species] = []

    by_species[species].append(name)
```
Let’s just track what happens after the first pet ({"name": "Flame", "species": "dragon"}):

species is "dragon", name is "Flame"
"dragon" not in by_species → by_species["dragon"] = []
Then by_species["dragon"].append("Flame")

Result after first pet:

Python

by_species == {
    "dragon": ["Flame"]
}
Does this step-by-step picture of the first iteration match what you expected?

Yes, it matches

Awesome 👌 Let's fast-forward through the next two pets so you see the full grouping.

We stopped here:

Python

by_species = {
    "dragon": ["Flame"]
}
Now second pet: {"name": "Spark", "species": "dragon"}

species → "dragon", name → "Spark"
"dragon" is already in by_species, so we skip creating []
We just do: by_species["dragon"].append("Spark")

Now:

Python

by_species = {
    "dragon": ["Flame", "Spark"]
}
Third pet: {"name": "Misty", "species": "unicorn"}

species → "unicorn", name → "Misty"
"unicorn" is not in by_species → create empty list: by_species["unicorn"] = []
Then append: by_species["unicorn"].append("Misty")

Final result:

Python

by_species = {
    "dragon": ["Flame", "Spark"],
    "unicorn": ["Misty"]
}
Before we practice in the editor:

Would you like me to show a small visual "buckets filling up" diagram for this, or are you ready to try coding this pattern yourself in the editor?

I'm ready to try coding

Nice, let's do it 💪

Here's what I'd like you to code in the Python editor (not in chat):

Create the pets list:
Python

pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Spark", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
]
Build by_species using the same logic we walked through:

start with by_species = {}
loop over pets
grab species and name
if the species key doesn’t exist yet, set it to an empty list []
append the name to the right list
at the end, print(by_species)

Then hit Run to check it, and when it works, hit Submit code so I can see it and give feedback.

Python

```
pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Spark", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
]
by_species = {}
for pet in pets:
    species = pet["species"]
    name = pet["name"]
    if species not in by_species:
        by_species[species] = []

    by_species[species].append(name)
print(by_species)
```

Nice work, Ron 🎉 That code is exactly the grouping pattern we wanted, and the output matches:

Python

```
{'dragon': ['Flame', 'Spark'], 'unicorn': ['Misty']}
```

Step you just mastered:

Start with an empty dict
For each item, create the bucket list if needed
Then .append() into that list
Next key idea for this lesson is choosing between:

list of dicts (pets = [...])
dict of lists (by_species = {...})

I’ll give you a small real-world scenario next, and you’ll tell me which structure fits better. Ready to compare them?

Yes, I'm ready

Cool, let’s compare with a simple scenario 👀

Imagine we’re building a search page for the shelter:

At the top, a big search bar: “type part of a pet’s name”
Below, we show all pets whose name matches the search

For this feature, we mostly need to scan every pet and check its "name".

In your gut, which structure sounds more natural for this feature?

A) List of dicts, one dict per pet: [{ "name": ..., "species": ... }, ...]
B) Dict of lists, one list per species: { "dragon": [...names...], "unicorn": [...names...] }
B dict of searchable keys

What do you think? Pick the option that best matches the search-by-name idea 👇

For the search-by-name feature (type a few letters, show all matching pets), which structure is usually a better fit?

A.

List of dicts like pets = [{"name": "Flame", "species": "dragon"}, ...]

B.

Dict of lists like by_species = {"dragon": ["Flame", ...], "unicorn": ["Misty", ...]}

a actually easier aces to names

Nice correction, Ron — A is the better fit here ☑

For a “search by pet name” feature:

With a list of dicts, you can just loop through every pet and look at pet["name"] directly. Simple scan.
With a dict of lists, names are split by species, so you’d have to jump through each species bucket to see all names – more steps, more nesting.

So:

Search across all pets by some field → list of dicts is usually clearer.
Quick lookup by a specific group/key (like species) → dict of lists shines.
Quick check-in:

Does this rule of thumb make sense?

“Search all rows” → list of dicts
“Jump straight to a bucket by key” → dict of lists
Yes, that helps

Great 😊

Now I want you to use this idea once yourself.

Think of these two shapes:

List of dicts (table of rows)

```
pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
]
```

Dict of lists (buckets)

```
by_species = {
    "dragon": ["Flame"],
    "unicorn": ["Misty"],
}
```

You’re building a page that, many times per second, needs to answer:

“Give me all pet names for a given species that a user selects from a dropdown.”

For this feature, which structure would you pick?

A) List of dicts (pets = [...])
B) Dict of lists (by_species = {...})
B for sure you can easily accss the species typed and have acess to the waned list of names

Exactly 💯

For “user picks a species, show all names in that species”:

Dict of lists is best because:
species → is the key
value → is already the exact list of names you need

So in code it’s basically:

Python

```
names = by_species[selected_species]
```

One direct hop. No scanning.

You’ve now done both big goals for this lesson:

Built a dict of lists incrementally to group by a key
Chosen between list of dicts vs dict of lists based on the feature

I’d like to give you one small practice task in the editor to lock it in.

Ready for a quick “you build the grouping” challenge with a different example (like grouping students by grade)?

I'm ready!

Love the energy 🔥

Let’s switch to a new example: students grouped by grade.

Step-by-step instructions for the Python editor (don’t write code in chat):

Make this list of students (exactly like this to keep it simple):
Python

```
students = [
    {"name": "Alice", "grade": "A"},
    {"name": "Bob", "grade": "B"},
    {"name": "Charlie", "grade": "A"},
    {"name": "Diana", "grade": "C"},
]
```

Build a dict called by_grade that ends up like:
Python

```
{
    "A": ["Alice", "Charlie"],
    "B": ["Bob"],
    "C": ["Diana"]
}
```

Use the same pattern as with pets:

start with by_grade = {}

loop over students
pull out grade and name
if this grade is not in by_grade yet, create an empty list []
append the name to the right list
print(by_grade) at the end

Run it, then hit Submit code so I can see it.

Python

```
students = [
    {"name": "Alice", "grade": "A"},
    {"name": "Bob", "grade": "B"},
    {"name": "Charlie", "grade": "A"},
    {"name": "Diana", "grade": "C"},
]
by_grade = {}
for student in students:
    grade = student["grade"]
    name = student["name"]
    if grade not in by_grade:
        by_grade[grade] = []

    by_grade[grade].append(name)
print(by_grade)
```

Beautifully done, Ron 🎯

Your by_grade code is spot on:

```
by_grade = {}
for student in students:
    grade = student["grade"]
    name = student["name"]
    if grade not in by_grade:
        by_grade[grade] = []
    by_grade[grade].append(name)
```

You've clearly nailed:

Building a dict of lists step by step
Choosing when to use list of dicts vs dict of lists
You've fully mastered this lesson ☑

Quick recap of what you can now do:

Use a list of dicts when you're scanning "all rows" (search/filter across everything)
Use a dict of lists when you want fast "jump to bucket by key" lookups (like species, grade, category)
You're ready to move on to the next lesson whenever you're re

```
pets = [
    {"name": "Flame", "species": "dragon"},
    {"name": "Spark", "species": "dragon"},
    {"name": "Misty", "species": "unicorn"},
```

```
]
by_species = {}
for pet in pets:
    species = pet["species"]
    name = pet["name"]
    if species not in by_species:
        by_species[species] = []

    by_species[species].append(name)
print(by_species)

students = [
    {"name": "Alice", "grade": "A"},
    {"name": "Bob", "grade": "B"},
    {"name": "Charlie", "grade": "A"},
    {"name": "Diana", "grade": "C"},
]
by_grade = {}
for student in students:
    grade = student["grade"]
    name = student["name"]
    if grade not in by_grade:
        by_grade[grade] = []

    by_grade[grade].append(name)
print(by_grade)
```